DOMINO Family Resilience Intelligence System

Technical report · Abu Dhabi University 6th Annual COE STEM Programming with AI Competition · Theme: AI for Family · Cycle 3 (Grades 9–12)

May Ahmed Mourad (G12) · **Shamsa Faris Almazrouei** (G12) · **Fatima Mohamed Almazrouei** (G11) — ADNOC Schools, Madinat Zayed · 28 October 2026

Prototype: maymourad.github.io/DOMINO **Source:** github.com/MayMourad/DOMINO *All households are fictional.*

Abstract. Family routines are chains of obligations: when one person becomes unavailable, a school pickup, a meal or a homework session can fail, and a child can be left without care. DOMINO models a household as a typed dependency graph, computes the direct and downstream consequences of a *hypothetical* disruption, searches for a legal recovery with Google OR-Tools CP-SAT, and re-checks every plan with a separate verifier before a person approves it. Language AI only proposes a structured event for human confirmation. On three fictional households, **46 of 51 benchmark cases pass and none fail** (5 are not run in the hosted environment); the flagship scenario is solved to proven optimality in a median of **133 ms**. We also report where the evidence is thin, including a tie with a greedy baseline on critical-task coverage.

1 Problem and approach

A shared calendar stores *events*, not the obligations and relationships behind them. Take the flagship household (the Amina family, four members, four tasks). Amina is unavailable from 14:30 to 18:00. Three tasks assigned to her break: the school pickup (critical, Tier 0), the grocery collection and the homework session. The pickup also *enables* the milestone "Yusuf arrived home", on which the homework depends, and Yusuf, aged nine, is a dependent who must only be collected by an authorized adult with a child restraint in the vehicle. Dinner, assigned to Khalid, is unaffected. A calendar shows one cancelled slot; DOMINO shows **4 direct and 2 indirect effects over three levels of depth**, and weighted task coverage falling from 100% to 25%, before it proposes anything.

DOMINO is built on five rules that we kept throughout. **(1) Simulate, do not predict:** only user-specified hypothetical events are analysed; no probabilities are produced. **(2) Language is separated from computation:** Gemini may propose an event; deterministic code resolves, validates and computes. **(3) Never invent capacity:** a task nobody can legally cover stays *uncovered* and is reported to a human. **(4) Check your own work independently:** a separate verifier re-derives every rule from raw data. **(5) Be reproducible:** one solver worker, a fixed seed and content hashes make identical inputs give identical plans. The system aligns in theme (only) with the UAE Year of Family 2026 and is not endorsed by any body.

What is new. (i) a typed, explainable dependency graph that carries a disruption to dependents and milestones, with every causal path kept; (ii) a lexicographic CP-SAT recovery planner whose unmet tasks stay visible instead of being forced; (iii) a verifier that is independent of the solver and checks 18 rules; (iv) a confirmation gate between language AI and computation; (v) an executable 51-case benchmark with honest baselines and stated limits.

2 System architecture

A React/TypeScript single-page application (GitHub Pages) calls a FastAPI service (Render, Docker) over a versioned `/v1` REST interface (Figure 1). The service layer is about 2,800 lines of Python with one module per responsibility; the browser holds no business logic beyond layout. The canonical *household snapshot* is the source of truth. Every edit in the browser invalidates in-flight requests and discards stale responses; the backend independently computes `scenario_hash = H(household, events)` and `input_hash = H(scenario_hash, rejections, pins)`, refuses recovery for a mismatched simulation (HTTP 409 `STALE_SCENARIO`) and re-verifies a plan against the current hash at approval. Errors are structured (`code, message, requestId, retryable`) and never include stack traces.

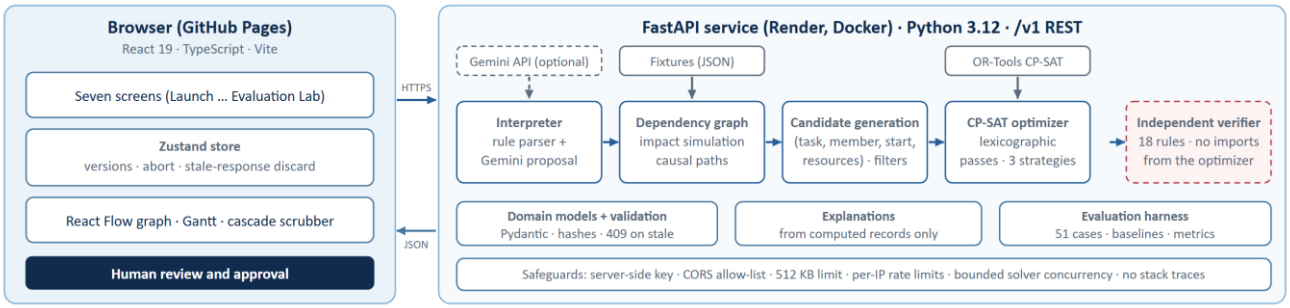


Figure 1. Architecture. The verifier (dashed, red) shares no code with the optimizer; Gemini is optional and can only propose events that a person must confirm.

3 Data model and typed dependency graph

A household (Pydantic, strict mode, referential and cycle checks) contains **members** (age, role, skills, explicit permissions, restrictions, availability, authorized dependents, a dependent flag), **tasks** (tier 0–3, priority weight, duration, time window, required skills, permissions and resource types, predecessors, dependents served, consent flag), **resources** (type, capacity, exclusive or shared, availability), **milestones**, a travel matrix on a 5-minute grid, and baseline assignments. Five disruption

types are supported: member unavailable, resource unavailable, task rescheduled, availability changed and resource capacity changed. Three fictional households (4 members each; 4–5 tasks) are shipped as validated fixtures, and each baseline passes the verifier with zero violations.

Table 1. Edge types of the dependency graph and how impact flows along them.

Edge type	From → to	Meaning	Impact flows?
`assigned_to`	task → person	who performs the task	yes, if the task is actually invalidated
`requires_resource`	task → resource	vehicle, kitchen, child restraint …	yes (resource events)
`requires_permission`	task → capability	eligibility: skill, permission, restriction	no (static filter)
`depends_on` / `enables`	task ↔ task, milestone	prerequisite order; completion achieves a milestone	yes (reversed / forward)
`serves_dependent`	task, milestone → dependent	care owed to a child or senior	terminal node
`disrupted_by`	entity → event	root cause of the cascade	root

Impact analysis. Starting from the confirmed event, a breadth-first propagation marks the targeted entity, then only those baseline tasks the event *actually invalidates* (interval intersection, window or capacity test, not mere assignment), then follows `enables` to milestones and reversed `depends_on` to dependent tasks, and ends at dependents served. Each node stores its depth, reason codes (for example `MEMBER_UNAVAILABLE`, `PREREQUISITE_THREATENED`) and *all* simple causal paths up to a cap, so any claim in the interface can be explained from computed records. After a plan exists every node is re-labelled recoverable, restored or unresolved on the backend.

4 Recovery optimization with CP-SAT

Candidates. For each task the engine enumerates tuples *(task, member, start, resource set)* on the 5-minute grid. Only *static* impossibilities are removed (skills, explicit permissions, restrictions, dependent authorization, a single member's or resource's availability, the task window); contention between tasks is left to the model. The flagship scenario yields **56 candidates** after filtering.

Model. One Boolean per candidate keeps eligibility and resource choice as static domain pruning while contention stays in standard global constraints:

```
x_c ∈ {0,1} for each candidate c = (t, m, s, R);   cov_t = Σ_{c∈C_t} x_c ≤ 1
member m:   NoOverlap( optional fixed-length intervals of all c with member m )
resource r: exclusive → NoOverlap ;  shared → Cumulative(demand_c ≤ capacity_r)
precedence: cov_t ⇒ cov_p  and  start_t ≥ end_p  (tasks, and tasks enabling milestones)
travel: candidates of one member closer than the travel matrix allows are exclusive
lexicographic: max Σ w_t·cov_t over Tier 0 → fix → Tier 1 → Tier 2 → Tier 3 → strategy terms
```

Why lexicographic. A weighted sum can trade one critical task for several minor ones; fixing each tier's optimum before the next pass guarantees that a Tier 0 task is never sacrificed for convenience. In the flagship run the five passes return Tier 0 = 10, Tier 1 = 12, Tier 3 = 2, displacement 0 and 3 changed assignments. Three strategies then differ only in their final preferences (Table 2); arrangements that coincide are shown once, and "another arrangement" adds a no-good cut instead of fabricating variety. The solver runs with one worker, seed 1 and a 2 s limit per strategy; the CP-SAT status (`OPTIMAL`, `FEASIBLE`, `INFEASIBLE` …) is kept separate from business completeness (`completeRecovery`, `criticalRecovery`, uncovered lists).

Table 2. Strategy preferences applied after coverage is fixed.

Strategy	Minimised after tier coverage, in order
Priority First	displacement from baseline time, number of changed assignments
Minimum Change	changed assignments, displacement, resource switches
Balanced Workload	maximum member load, load spread, changes, displacement

Table 3. Key engineering decisions and their cost (full record: `docs/decisions/0001-key-decisions.md`).

Decision	Reason	Cost
One Boolean per (task, member, start, resources)	eligibility and resource choice become static pruning; contention stays in NoOverlap / Cumulative	more variables (hundreds at household scale)
Lexicographic passes, not a weighted sum	a Tier 0 task is never traded for several minor tasks	several solves per strategy (tens of ms each)
Resource model: capacity plus an exclusive flag	vehicles are exclusive; the pickup needs 2 seat-units, so "seats reduced to 1" makes it infeasible	demands are declared by hand
Verifier shares no code with the optimizer	a modelling bug cannot validate itself	rule logic is duplicated on purpose
Gemini only extracts	code resolves and validates; every event starts unconfirmed	relative phrases need household context

Flagship result. DOMINO moves the pickup to Khalid (14:45–15:25, family sedan plus child restraint), the grocery collection to Khalid (15:55–16:30) and the homework session to Noura (16:45–17:15); dinner is untouched. The pickup is flagged as pending consent. All three strategies produced this single arrangement, which the interface states instead of showing three copies (Figure 2).

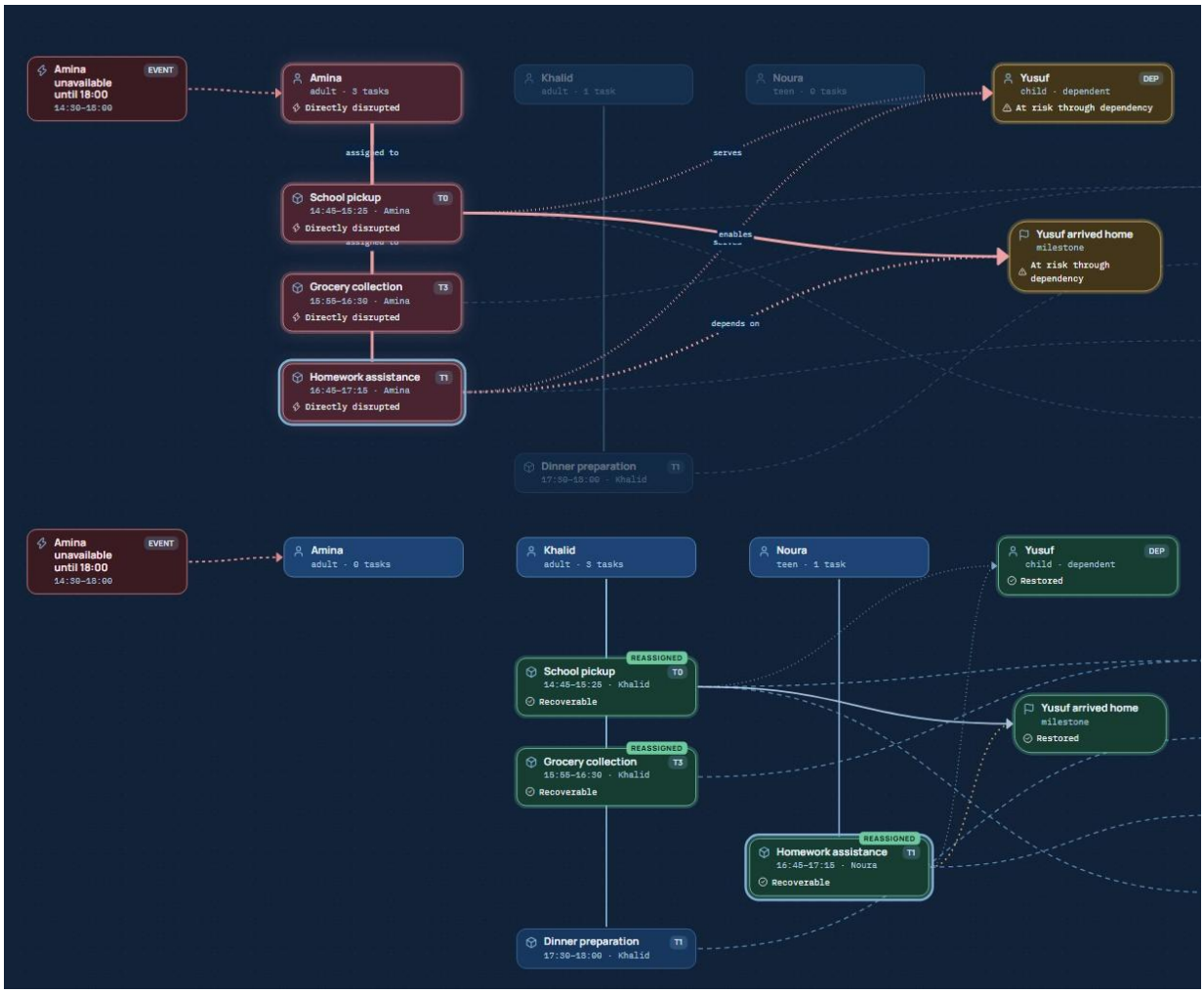


Figure 2. Real screenshots of the prototype for the flagship scenario. Top: impact graph (red = directly disrupted, amber = at risk through dependency). Bottom: the verified recovery (green = reassigned or restored). Resource nodes at the right edge are cropped for legibility.

5 Independent verification and honest failure

`verifier.py` imports nothing from the optimizer or candidate generator; the rule logic is deliberately duplicated so that a modelling bug cannot validate itself. It recomputes 18 rules (Table 4) directly from the household and reports each with its evidence. Plans carry three separate labels: *assignment validity*, *recovery completeness* and *approval*; a plan with any failed hard check is never labelled valid, and approval additionally requires user confirmation, consent acknowledgement, no unresolved critical task and a matching hash. The flagship plan passes **56/56** checks. Negative tests inject unauthorized driving, overlaps, a double-booked vehicle, a missing prerequisite, unavailability, capacity violations, a stale hash and misreported coverage, and the verifier rejects each.

Table 4. The 18 verifier rules, grouped.

Group	Rules	What is re-derived from raw data
Eligibility	V03–V05, V12	skills, explicit permissions, restrictions, eligibility list, dependents are never assigned, only authorized adults care for a dependent
Time and travel	V06–V08, V14	duration and window, member availability, no overlaps including travel gaps, itinerary transitions
Resources	V09–V11	availability, exclusivity and capacity, required resource types satisfied by permitted resources
Structure	V01, V02, V13	references exist, one primary assignment per task, prerequisites and milestones precede dependants
Governance	V15–V18	scenario hash, accurate critical-coverage report, consent stays pending, user rejections and pins honoured

When recovery is impossible. If Khalid is also unavailable from 14:40 to 15:45, no legal assignee exists for the pickup (Noura lacks the driving skill and the pickup permissions, carries driving restrictions, and Yusuf is a dependent). DOMINO returns a *valid partial plan*: solver status `OPTIMAL`, 30/30 checks passed, weighted coverage 33%, the pickup and the homework session listed as uncovered critical tasks and flagged for human intervention. It does not invent a caregiver (Figure 3).

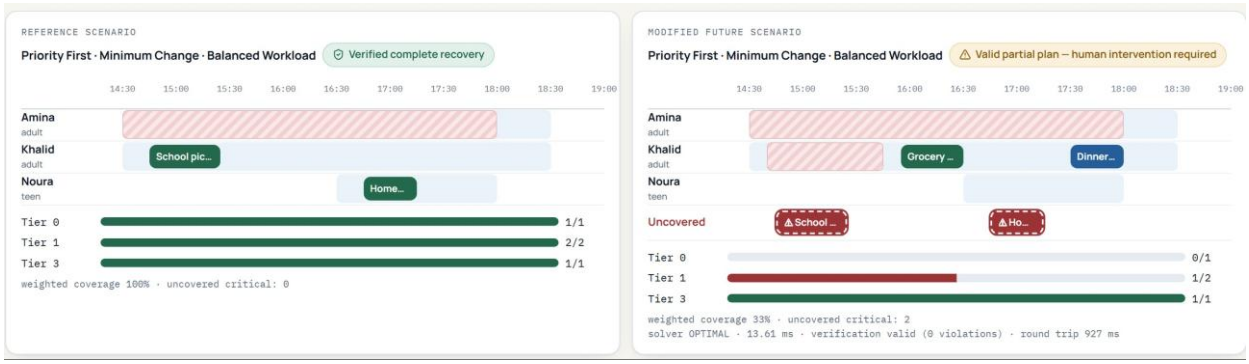


Figure 3. Real screenshot of the Futures Laboratory: the reference scenario (left, complete recovery) and the modified scenario (right, valid partial plan with two uncovered critical tasks shown as dashed red blocks). Hatched bars are unavailability.

6 Natural-language input and responsible AI

A user may type “Amina is unavailable until 6 PM”. Two interchangeable engines turn this into a *proposed, unconfirmed* event: a deterministic rule parser (always available) and Gemini (`gemini-3.5-flash-lite` through `google-genai`, structured JSON output against a Pydantic schema). The model returns entity mentions and HH:MM strings only. The backend then resolves names against the current household (duplicates become ambiguity, unknown names become unresolved), validates windows, refuses to guess a start for “until 6 PM” without a scenario clock, flags instruction-like text in the input and always emits `confirmed = false`. Automatic mode falls back to the rule parser on any Gemini failure. The key lives only on the server; the published JavaScript was scanned and contains no key pattern.

Table 5. Interpretation accuracy on two 12-sentence sets (hosted API, 28 Oct 2026).

Engine	Annotated set	Held-out set
Built-in rule parser	12/12	12/12 (see caveat)
Gemini `gemini-3.5-flash-lite`	12/12	11/12

Caveats. The rule parser first scored 11/12 on the held-out set and was then fixed, so its 12/12 is no longer independent. Gemini’s miss is “Dinner is delayed by 30 minutes”, which needs a task’s baseline time that only the rule parser has in code. Earlier Gemini runs on the annotated set varied (83%, 92%, then 100% three times) before the prompt was tightened. Twelve sentences per set is a sanity check, not an accuracy claim.

7 Implementation, deployment and security

Table 6. Technology stack and deployment.

Layer	Technology	Notes
Frontend	React 19, TypeScript, Vite, Zustand, React Flow (`@xyflow/react`), Motion	hash routing so refresh works on GitHub Pages; about 3,300 lines of TypeScript; original canvas “constellation” view
Backend	Python (3.12 hosted), FastAPI, Pydantic v2, NetworkX	versioned `/v1` REST, structured errors, OpenAPI at `/docs`
Optimization	Google OR-Tools CP-SAT 9.15	one worker, seed 1, 2 s limit per strategy
Language AI	`google-genai` with `gemini-3.5-flash-lite`; built-in rule parser	server-side key only; interpretation limited to 12 requests per minute per IP
Hosting	GitHub Pages (frontend) · Render free tier, Docker (API)	GitHub Actions: lint, tests, build, e2e; auto-deploy on push to `main`
Safeguards	CORS allow-list (the Pages origin), 512 KB body limit, per-IP rate limits, solver concurrency 4	no secret in the repository or the published bundle; no stack traces

The prototype. Seven screens follow the analysis pipeline (Figure 4): *Launch* (household choice, service status), *Family Network* (the typed graph and an inspector, with household editing), *Disruption Studio* (presets, a manual form and natural-language input with a review step), *Cascade Laboratory* (step-by-step impact and causal paths), *Recovery Studio* (plans, timeline, verification, approval), *Futures Laboratory* (a reference and a modified scenario side by side) and *Evaluation Lab* (the benchmark, baselines and methodology). Plans can be exported as JSON, as a calendar file or as a printable page. Approval only records a simulation receipt: DOMINO never messages, books or instructs anyone.

AI-assisted development. The prototype was built with AI assistance (Claude Code, Claude Sonnet 5.5, in the Claude desktop app); sessions, decisions and corrections are indexed in `AI_DEVELOPMENT_LOG.md`, and the full chats are submitted as a separate item. Defects found by review or live testing and then fixed include an unfair baseline comparison (the greedy baseline ignored user rejections), a React Flow fit-view that did nothing because node sizes were not fed back, an alias “car” that matched two resources, a `GEMINI_MODEL` set to a project id (now detected and reported), provider errors misread as rate limits, a navigation race after a hosted household loaded and a scenario clock that was shown but not sent.

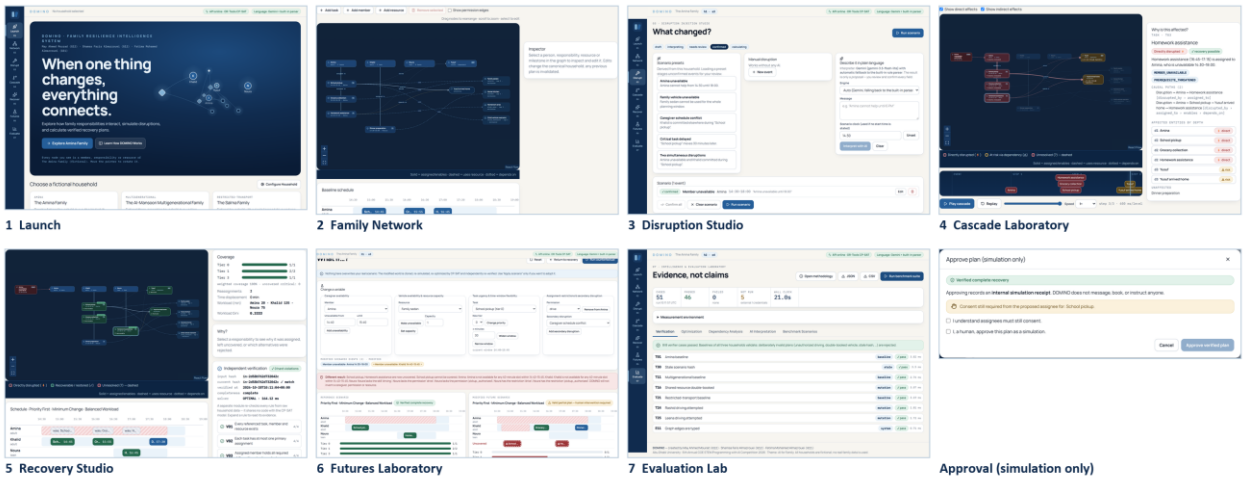


Figure 4. The seven screens of the published prototype and the approval dialog (real screenshots, cropped to the top of each screen).

8 Evaluation

Method. Each of the 51 benchmark cases (T01–T30 from the specification, E01–E17 and X01–X04) loads a fixture, applies declared edits and events, runs the real engine and checks declared invariants with the independent verifier; cases contain no recorded outcomes. Baselines are *no recovery* and *greedy first-fit* (by tier, then weight, respecting prerequisites), both also passed through the verifier. Dependency detection is scored as precision, recall and F1 of the affected set against hand-written expectations. Workload balance uses the Gini coefficient of member minutes. Each saved run records Python, OR-Tools, CPU count and solver settings.

Table 7. Results recorded on the hosted API, 28 Oct 2026 (Linux, 8 CPUs, OR-Tools 9.15, one worker, seed 1).

Measure	Result	Reading and limitation
Benchmark cases	51 defined · **46 pass** · 0 fail · 5 not run	4 frontend cases run in Playwright/build; 1 AI case is skipped once a key is configured
Dependency detection (11 cases)	P = R = F1 = 1.00	small, hand-authored set: correctness on these cases only
Flagship solver time	median 133 ms (range 43–167, 10 runs)	free-tier CPU; engineering target is 2 s per strategy
Per-case wall time (27 solves)	median 802 ms · p95 1700 ms	includes three strategies, baselines and verification
HTTP round trip, flagship	≈ 0.5 s simulate · ≈ 0.9 s recover	includes network; varies by day

Table 8. Software test layers (commands and results are recorded in `TESTING.md`).

Layer	Scope	Result
Backend (pytest)	schemas, graph propagation, optimizer, verifier negative tests, interpreter safety, API pipeline, the 51-case suite	63 passed; ruff clean
Frontend (Vitest)	time helpers, typed edges, stable layout, stale-response discard, invalidation, undo	11 passed; type-check and lint clean
End to end (Playwright)	flagship 12-step flow, other households, approval, refresh, offline, keyboard, language input, edits, export	15 passed locally; on the live site 14 on the first run, the 15th passed 5/5 after a wait fix
Live smoke and CI	hosted health, CORS, simulate and recover, stale hash, determinism, Gemini, benchmark	all passed; CI green on Ubuntu

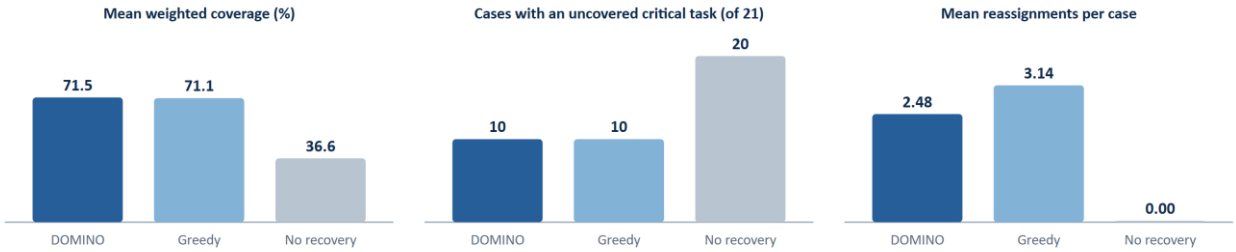


Figure 5. Baseline comparison over the 21 disruption cases. All baseline outputs were independently verified valid.

Reading the baselines honestly. Against doing nothing, recovery roughly doubles weighted coverage (71.5% vs 36.6%) and halves the cases with an uncovered critical task (10 vs 20 of 21). Against greedy first-fit, DOMINO's mean coverage is only 0.4 percentage points higher and **the two tie on critical coverage in every one of the 21 cases**; its measurable advantage on these small households is fewer changed assignments (2.48 vs 3.14 per case), plus the lexicographic guarantee and deterministic behaviour. We expect separation to grow with larger, more contended households, but we have not measured that.

9 Limitations, responsible use and next steps

- **Scope of evidence:** synthetic households of four members; hand-authored expectations; one simulated day; no user study and no measurement of wellbeing. The verifier checks the model's declared rules, not real-world safety.
- **AI:** natural-language extraction can be wrong, so confirmation is mandatory; the sentence sets are small and the rule parser's held-out score is not independent. The language model never schedules, validates or approves.
- **Operations:** the free-tier backend sleeps when idle, so the first request can take 30–60 s; the prototype can also be run locally from the repository. Strategies often collapse into one arrangement on the flagship household.
- **Not built:** animations in the deck, a robustness score, a resilience index, real calendar or messaging integration, a full WCAG audit, Safari and Firefox testing.
- **Next steps:** larger and multi-generational households, travel-time estimation, multi-day horizons, Arabic input, usability studies with families, and privacy-preserving data handling.

Reproducibility and sources

Everything above can be re-run: `POST /v1/benchmarks/run`, `pytest backend/tests/benchmarks`, `python scripts/live_smoke.py <api>` and the Playwright suite; saved runs are in `docs/benchmark-results/`. Methodology: `EVALUATION_METHODOLOGY.md`; decisions: `docs/decisions/0001-key-decisions.md`; attribution of libraries and visual references: `docs/references/ATTRIBUTION.md`. External references: Google OR-Tools CP-SAT documentation (developers.google.com/optimization/cp/cp_solver); Gemini structured output (ai.google.dev/gemini-api/docs/structured-output); React Flow; FastAPI; UAE Year of Family 2026 (u.ae/en/Year-of-family/Family-year-2026). AI-assisted development is indexed in `AI_DEVELOPMENT_LOG.md`.